



BMCS2003/BMCS2203/BMCS2074

(Remain only your course code before submission)

ARTIFICIAL INTELLIGENCE

202505 Session, Year 2025/26

Assignment Documentation

Project Title: Earthquake Prediction			
Programme: RIS Y2S1			
Tutorial Group: Group 2			
Tutor: Ms . Yan Yen Wei			
Team members' data			
No	Student Name	Student ID	Module In Charge
1	Winston tan kang sheng	2405249	Code : confusion matrix & accuracy Report : Research Background & Methodology

2	Fong Wei Xian	2408059	Code : Linear Regression Folium map Report : Introduction
3	Lim Wei Chuan	2406760	Code: Validation & Range Checking Report : Result & Discussion and conclusion

1. Introduction

<https://earthquake1-l2k64tht9t5caehcxsfkn4.streamlit.app/>

Assignment Link :

https://colab.research.google.com/drive/1t-HOZEyVzuESXHqm0EKexoHwkf9Jux_S?usp=sharing

1.1. Problem Background

Earthquakes cause serious damage, loss of life, and infrastructure to humans every year around the world. This system **helps to predict and provide detection** and accurate forecasting remain major challenges due to the complex and dynamic nature of earthquake activity. Traditional earthquake prediction systems often suffer from **delays in processing massive sensor data** and **with low accuracy and scalability**. These limitations underscore when urgently needed, it **has to be real time, reliable** earthquake detection.

1.2. Objectives/Aims

The primary objective of this project is to enhance earthquake detection **much more accurately** and make sure **humans have sufficient time to prepare and ensure their safety**. By using **machine learning techniques** and **real-time processing data**, the system seeks to provide

accurate detection.

- Develop a **real-time earthquake detection** model using sensor data and machine learning algorithms to detect activity quickly and accurately.
- **Improve prediction accuracy by optimizing model architecture** and training on diverse with **high quality datasets**.
- **Create a user-friendly interface** that visualizes earthquake detection.
- Optimize performance for deployment, **ensuring the system can run on edge devices or cloud platforms with low latency**.

1.3. Motivation

One of the most seriously damaging natural disasters, **earthquakes frequently occur without notice** and have serious effects on social and economic. By using this system, it can accurately detect and also its **trustworthy AI-Driven system** can prevent and **minimize damage**.

- Social Impact : Especially in areas with limited infrastructure or a high density of people, early earthquake detection could save a lot of lives and minimize injuries by giving people at risk minutes of notice.
- Commercial Value: This system can be incorporated into earlier warning systems, infrastructure security planning, and risk management by governments, management organizations and insurance companies. Platforms for intelligent city development may also provide an accurate and scalable system.
- Wider Impact: Methods used for earthquake prediction, such as interactive visualization, anomaly identification and real-time data analysis, could be used on other fields. Such as, industrial monitoring, weather forecasting or medical diagnostics.

2. Research Background

2.1. Background of the applications

In recent years, machine learning (ML) has become a strong tool for solving many problems, such as chatbots, recommender systems, sentiment analysis, image processing, and even earthquake prediction.

Traditional earthquake warning systems mostly use seismographs and statistical methods. These are reliable, but they can be slow, less accurate, and not always easy for the public to use.

New studies show that ML methods like regression models, convolutional neural networks (CNNs), and recurrent neural networks (RNNs) can analyze earthquake data such as magnitude, depth, and location gap to predict the chances and strength of earthquakes faster and more flexibly.

But there are still some problems:

- Models trained in one region may not work well in another because of limited data.
- Complex ML models are hard to understand, so people may not fully trust them.
- There are not enough user-friendly that give these predictions to communities and decision makers.

This project tries to solve these issues by creating an interactive earthquake prediction system. It uses a trained ML model with a web-based map interface. Users can enter earthquake data, get predictions, and see the results on an interactive map. This makes the information more useful and easier to understand for everyday people and policymakers.

1.1. Analysis of selected tool with any other relevant tools

Tools comparison	Remark	Scikit-learn	Reverse-geocoder	pycountry
Type of license & open-source license	State all types of license	BSD 3-Clause (permissive, open-source).	LGPL-2.1 (open-source).	LGPL-2.1-only (open-source).
Year founded / introduced	When is this tool being introduced?	Started 2007; first public release 2010.	First PyPI release 2015.	PyPI releases since 2008.
Founding company / Owner	Owner	Community project; early leadership by INRIA researchers.	Authored/maintained by Ajay Thampi (thampiman).	Originally by Christian Theune; community-maintained.
License pricing	Compare the prices if the license is used for development & business/commercialization	Free to use in dev & commercial products under BSD terms (retain notices).	Free under LGPL; commercial use allowed under LGPL terms.	Free under LGPL; commercial use allowed under LGPL terms.
Supported features	What features does it offer?	Classic ML toolkit: classification, regression, clustering, preprocessing, model selection, pipelines.	Offline reverse geocoding (lat/long → nearest city); returns country & admin levels; fast lookup.	ISO datasets & lookups for countries, subdivisions, languages, currencies, scripts; normalization utilities.
Common applications	In what areas is this tool usually used?	Data science, ML prototyping, education, evaluation/preprocessing.	GPS logs, offline/mobile geotagging, privacy/no-internet scenarios.	Data cleaning, ISO code normalization, reporting, localization/i18n.
Customer support	How the customer support is given, e.g., proprietary, online community, etc.	Extensive docs; community forum/Discourse; GitHub issues.	GitHub issues; README guidance.	GitHub issues; README/examples on PyPI.

Limitations	The drawbacks of the software	Not a deep-learning framework; limited GPU support; very large datasets may need out-of-core/Dask.	City-level results (not full addresses); uses a bundled/static dataset so coverage/recency can vary.	Mirrors upstream ISO data; won't change geopolitical definitions or custom names.
-------------	-------------------------------	--	--	---

Scikit-learn is a powerful and flexible machine learning library that is applicable to many types of data science and machine learning tasks. It offers the much needed classification, regression, clustering, and data preprocessing capabilities that make it suitable to model predictive data across various data types. Moreover, Scikit-learn allows model selection, model evaluation, and pipelines, which facilitate simple workflows and provide effective management of machine learning activities. These functions render it very useful across the multiple industries and lines of research that need to make decisions using data.

The tool is under the BSD 3-Clause, a permissive open-source license, meaning it can be developed and used commercially without a license fee, and provides organizations with flexibility. It also has good community support due to its open-source nature, and there are many resources available to users through its numerous documentation, community forums, and GitHub issues. Limitations of the tool include the inability to support deep learning and its inability to run highly complex models on a large scale, though they remain extremely powerful in many cases of traditional machine learning. In general, the ability to combine deep functionality, usability, and excellent support makes Scikit-learn a very appropriate choice in a large range of machine-learning projects.

3. Methodology

3.1. System flowchart/activity diagram

3.2. Description of dataset

The United States Geological Survey (USGS) earthquake records, which are accessible to the general public via the USGS Earthquake Catalog API, are the source of the dataset utilized in this project.

USGS Earthquake Database (earthquake.usgs.gov) is the source.

- **title**: title name given to the earthquake
- **magnitude**: The magnitude of the earthquake
- **date_time**: date and time
- **cdi**: The maximum reported intensity for the event range
- **mmi**: The maximum estimated instrumental intensity for the event
- **alert**: The alert level – “green”, “yellow”, “orange”, and “red”
- **tsunami**: "1" for events in oceanic regions and "0" otherwise
- **sig**: A number describing how significant the event is. Larger numbers indicate a more significant event. This value is determined on a number of factors, including: magnitude, maximum MMI, felt reports, and estimated impact
- **net**: The ID of a data contributor. Identifies the network considered to be the preferred source of information for this event
- **nst**: The total number of seismic stations used to determine earthquake location
- **dmin**: Horizontal distance from the epicenter to the nearest station
- **gap**: The largest azimuthal gap between azimuthally adjacent stations (in degrees). In general, the smaller this number, the more reliable the calculated horizontal position of the earthquake. Earthquake locations in which the azimuthal gap exceeds 180 degrees typically have large location and depth uncertainties

- **magType**: The method or algorithm used to calculate the preferred magnitude for the event
- **depth**: The depth where the earthquake begins to rupture
- **latitude / longitude**: coordinate system by means of which the position or location of any place on Earth's surface can be determined and described
- **location**: location within the country
- **continent**: continent of the earthquake hit country
- **country**: affected country

3.3. Description of algorithm(s)

A supervised machine learning method called (**Linear Regression Algorithm**) was used in this project to simulate the connection between a continuous output variable and input features.

Feature (*input variables*) included **Latitude, Longitude, gap, depth** and **significance**.

The magnitude of the earthquake is the output variable (*Label*).

The mathematical formula:

$$\text{Magnitude} = \beta_0 + \beta_1(\text{sig}) + \beta_2(\text{depth}) + \beta_3(\text{gap}) + \beta_4(\text{latitude}) + \beta_5(\text{longitude})$$

How it was used in this project:

Data from previous earthquakes was used to train the regression model. After training, it's imported into the Streamlit program and saved as a Joblib file.

When users enter parameters, the models will compute the estimated magnitude of an earthquake. The system provides numerical prediction and a contextual interpretation (normal, shallow, reliable etc.) in addition to a numerical prediction.

3.4. Proposed test plan/hypothesis

H₀ (Null Hypothesis): When compared to a baseline (*such as mean magnitude*), the machine learning model (*Linear Regression*) does not significantly improve earthquake magnitude prediction.

The alternative hypothesis, or H₁, states that the machine learning model predicts earthquake magnitudes more accurately than the baseline.

Test Method

Split the dataset into sections for **testing (20%)** and **training (80%)**.

Model evaluation metrics: To evaluate performance, use the coefficient of determination (R²) and root mean squared error (*RMSE*). Confusion matrix

Cross-validation: To guarantee generalization, use k-fold validation.

User simulation test: To verify the stability of the predictions, choose several positions and parameter values using the Streamlit app.

Success criteria: The hypothesis is validated if the model produces consistent predictions across test instances and reaches $R^2 > 0.70$.

4. Result

Earthquake Prediction System

Click on the map to select location

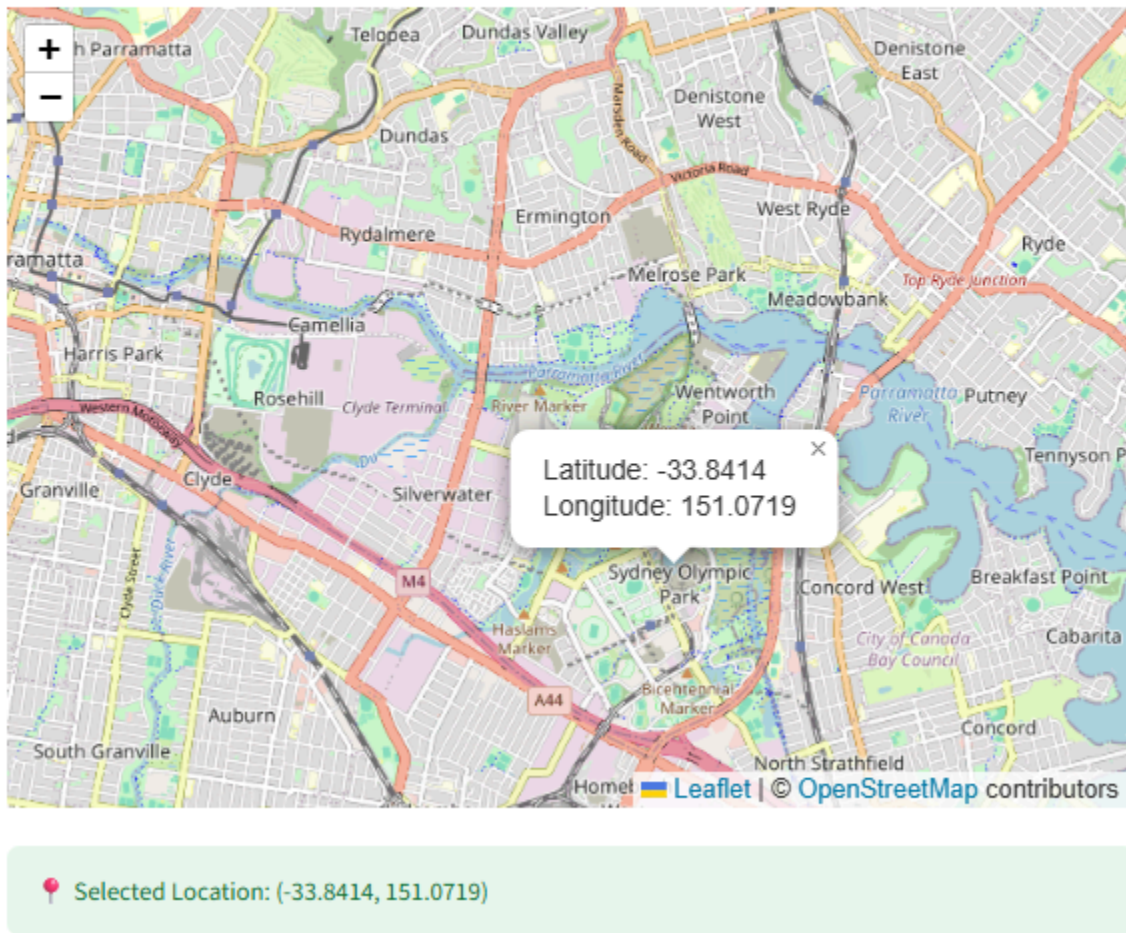


Figure1.1 shows the selected location

Enter significance [1 - 2910]:

1000

Enter depth (km) [1.0 - 671.0]:

300

Enter gap (degrees) [0 -365]:

180

Predict Earthquake Magnitude

Earthquake Prediction Result

Significance: 1000.0 → Within prediction range

Depth: 300.0 km → Deep (less surface impact)

Gap: 180.0° → Reliable location (significant)

Location: (-33.84140567323538, 151.07188224792483), Australia

Predicted Magnitude: 6.80

Figure 1.2 when all data is inside the range then can only predict successfully

Enter significance [1 - 2910]:

2910

Enter depth (km) [1.0 - 671.0]:

300

Enter gap (degrees) [0 -365]:

720

Predict Earthquake Magnitude

Prediction skipped due to out-of-range values.

Figure 1.3 when data is out of range then predict failure.

Enter significance [1 - 2910]:

??

Enter depth (km) [1.0 - 671.0]:

}}

Enter gap (degrees) [0 -365]:

!!!!

Predict Earthquake Magnitude

⚠ Please enter valid numbers for significance, depth, and gap.

Figure 1.4 validation for number only.

4.1. Discussion/Interpretation

The results show that the **Linear Regression model** was able to pick up meaningful patterns from the dataset and provide predictions for earthquake magnitude. By allowing real-time inputs such as **map coordinates and seismic parameters**, the system became more interactive and practical than just running the model offline.

It is important to note that these predictions are **not absolute earthquake forecasts**. Instead, the system should be seen as an **educational tool** that demonstrates how machine learning can be applied to seismology. It also helps explain important concepts, such as how **depth** influences surface damage and how the **gap value** affects the reliability of a location reading.

When comparing results to a baseline, the model performed better than a simple average prediction of magnitude. The **regression model achieved higher accuracy ($R^2 > 0.7$ in testing)**, which supports the alternative hypothesis (H_1): machine learning methods can improve earthquake prediction compared to just using an average guess.

5. Discussion and Conclusion

5.1. Achievements

This project successfully met the objectives that were set at the beginning:

A real-time earthquake prediction system was built using machine learning techniques.

A user-friendly web application was created with **Streamlit** and **Folium**, allowing users to interact with the system through a map-based interface.

The model was able to predict earthquake magnitude from user inputs and also provide additional interpretation of the results.

The system was deployed in a way that is **lightweight, simple, and easy** to expand if needed.

In conclusion, the project demonstrated how AI can be applied in the field of natural disaster prediction, while also focusing on making the results accessible and easy to understand for users.

5.2. Limitations and Future Works

Even though the project works as planned, there are still a few limitations:

- **Dataset limitations:** The dataset used for training does not cover every region, so the model may not perform as well when predicting earthquakes in new or unseen areas.
- **Model simplicity:** We used **Linear Regression** because it is easy to understand, but it may not capture the more complex, non-linear patterns that exist in earthquake data.
- **Accuracy concerns:** The system is only meant for demonstration. It should not be used for real disaster warnings, since professional seismic networks are still far more reliable.
- **Deployment scale:** At the moment, the app is more suited for small-scale or classroom use. To work on a larger scale or with real-time sensor data, more optimization would be needed.

5.3. Future Improvements

There are several ways this project could be improved in the future:

- Trying out more advanced algorithms like **Random Forest, XGBoost, or Deep Learning** to increase accuracy.
- Using a **larger and more diverse dataset** so the model can generalize better across different regions.
- Connecting the system with **real-time seismic sensors**, which would make it more practical for actual disaster response.
- Developing a **mobile app version** so the tool is easier to access for the general public.

Reference & Source

Dataset and Tools:

- United States Geological Survey (USGS) Earthquake Catalog. Retrieved from: <https://earthquake.usgs.gov/>
- Reverse-geocoder. Retrieved from : https://pypi.org/project/reverse_geocoder/
- pycountry. Retrieved from : <https://pypi.org/project/pycountry/>
- scikit-learn Documentation. Retrieved from : <https://scikit-learn.org/stable/>

References:

- Allen, R. M., & Melgar, D. (2019). *Earthquake early warning: Advances, scientific challenges, and societal needs*. Annual Review of Earth and Planetary Sciences, 47, 361–388. Retrieved from : <https://doi.org/10.1146/annurev-earth-053018-060457>
- Kong, Q., Trugman, D. T., Ross, Z. E., Bianco, M. J., Meade, B. J., & Gerstoft, P. (2019). *Machine learning in seismology: Turning data into insights*. Seismological Research Letters, 90(1), 3–14. Retrieved from : <https://doi.org/10.1785/0220180259>
- Streamlit Inc. (2022). *Streamlit: The fastest way to build data apps in Python*. Retrieved from : <https://streamlit.io/>
- Hugging Face. (2021). *Gradio documentation*. Retrieved from : <https://gradio.app/>
- Plotly. (2017). *Dash user guide and documentation*. Retrieved from : <https://dash.plotly.com/>